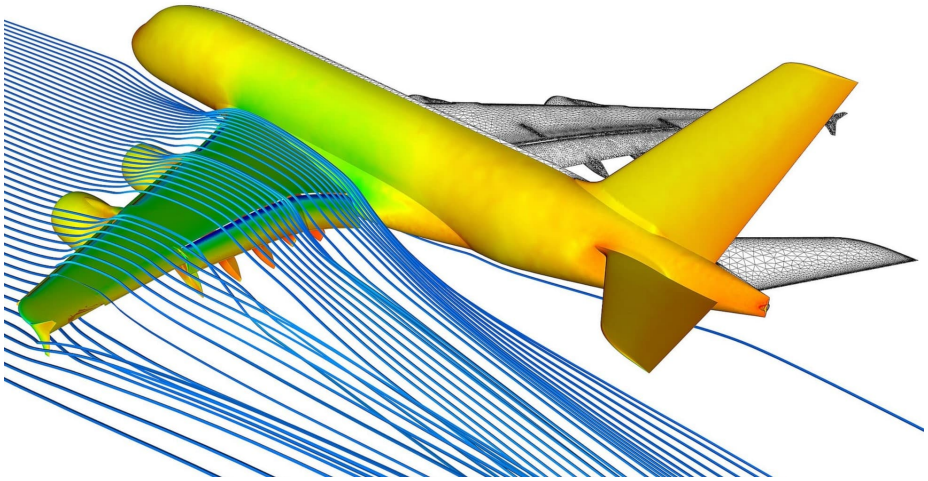
ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

Τμήμα Μηχανολόγων Μηχανικών

ΕΡΓΑΣΙΑ ΑΕΡΟΔΥΝΑΜΙΚΗΣ

Αεροδυναμικά Χαρακτηρηστικά Αεροτομής NACA 0012 με Μεταπτερύγιο (Flap)



Μάθημα: Αεροδυναμική
Φοιτητής: Marios Petropoulos
Α.Μ.: mc22043

Ακαδημαϊκό Έτος 2025–2026

Εισαγωγή

Η παρούσα εργασία έχει ως στόχο τη μελέτη της αεροδυναμικής συμπεριφοράς μιας πτέρυγας πεπερασμένου εκπετάσματος με μεταπτερύγιο μέσω χρήσης αριθμητικών μεθόδων. Συγκεκριμένα, πραγματοποιείται η μέθοδος γραμμής άνωσης του Prandl εφαρμόζοντας την δυναμική συνθήκη.

Η μέθοδος αυτή βασίζεται στην υπόθεση ασυμπίεστης, ιδανικής ροής και επιτρέπει την προσεγγιστική επίλυση του προβλήματος ροής γύρω από πτέρυγες πεπερασμένου μήκους μέσω διακριτοποίησης της γεωμετρίας σε πεπερασμένο αριθμό δινοπετάλων.

Στο πλαίσιο της εργασίας αναπτύχθηκε υπολογιστικός κώδικας σε γλώσσα C++, ο οποίος υπολογίζει τις επαγόμενες ταχύτητες μέσω του νόμου Biot–Savart, επιλύει το σύστημα εξισώσεων για την κυκλοφορία και εξάγει τον συντελεστή άνωσης C_L συναρτήσει της γωνίας προσβολής α για διάφορες τιμές τις γωνίας κλίσης του μεταπτερυγίου (flap) δ_f.

1 Δεδομένα του προβλήματος

Η υπό μελέτη πτέρυγα στο συγκεκριμένο πρόβλημα έχει τα εξής χαρακτηριστικά :

- Αεροτομή NACA 0012
- Λόγο επιμήκους $AR = 7$
- Μήκος χορδής $c = 1$

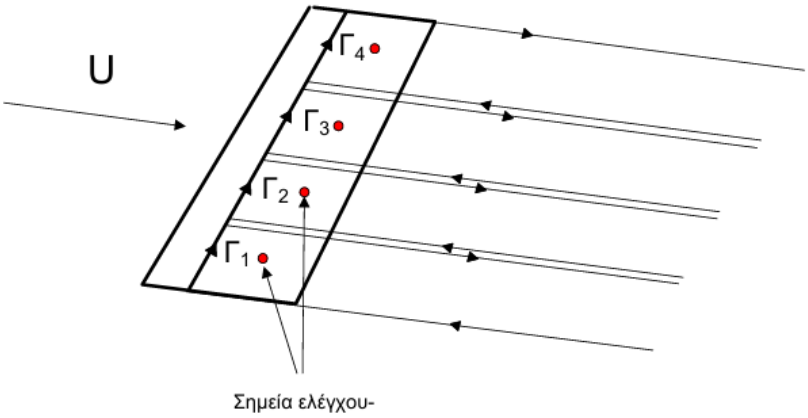
Συνεπώς, μιλάμε για μία συμμετρική αεροτομή, με γωνία μηδενικής άνωσης $\alpha_0 = 0$ και μήκος εκπετάσματος $b = AR * c = 7$.

Όσον αφορά τις συνθήκες πτήσης, λαμβάνουμε αυθαίρετα μια ρεαλιστική ταχύτητα $U = 100m/s$.

2 Ορισμός γεωμετρίας και Διακριτοποίηση

Η πτέρυγα αποτελείται από το κύριο τμήμα (main part) και το μεταπτερύγιο (flap). Η γεωμετρία της ορίζεται σε καρτεσιανό σύστημα συντεταγμένων, με τον άξονα x παράλληλο στη διεύθυνση της ελεύθερης ροής και τον άξονα y κατά το εκπέτασμα της πτέρυγας ενώ το κέντρο των αξόνων βρίσκεται στο LE στο μέσο της πτέρυγας. Για λόγους απλοποίησης, θεωρείται ότι όλα τα σημεία της πτέρυγας βρίσκονται στο ίδιο επίπεδο (z=0). Το κύριο τμήμα και το μεταπτερύγιο διακριτοποιούνται ξεχωριστά κατά το εκπέτασμα σε πεπερασμένο αριθμό στοιχειωδών πάνελ ίσου μήκους. Κάθε πάνελ αναπαρίσταται από έναν δεσμευμένο στρόβιλο τοποθετημένο στη θέση $x = c/4$ και ένα σημείο ελέγχου (control point)

στη θέση $x = 3c/4$. Από τα άκρα κάθε δεσμευμένου στροβίλου εκκινούν ουραίοι στρόβιλοι που εκτείνονται κατάντη προς το άπειρο.

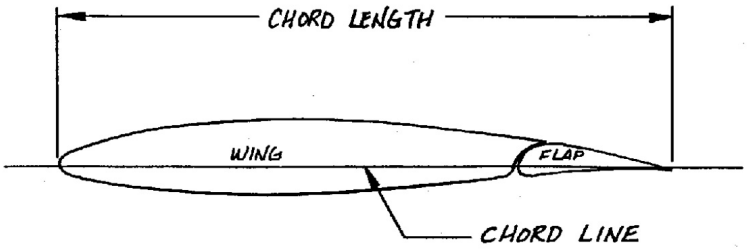


Σχήμα 1: Σχηματική αναπαράσταση της διακριτοποίησης κάθε part της πτέρυγας

Όσον αφορά το main part της πτέρυγας, αυτό εκτείνεται ως το $x = 0.75c$, από όπου ξεκινάει το flap μέχρι το τέλος της πτέρυγας ($x = c$). Ορίζονται λοιπόν οι επιμέρους χορδές κάθε part της πτέρυγας ως εξής:

- $chord_{main} = 0.75c$
- $chord_{flap} = 0.25c$

Το μεταπτερύγιο θεωρείται ότι βρίσκεται υπό γωνία κλίσης δ_f ως προς το κύριο τμήμα της πτέρυγας. Η γωνία αυτή λαμβάνεται υπόψην κατά την επιβολή της συνθήκης μη διείσδυσης της ροής στα σημεία ελέγχου. Ενδεικτική εικόνα της διάταξης των parts της πτέρυγας φαίνεται στο παρακάτω σχήμα:



Σχήμα 2: Σχήμα Πτέρυγας με flap

3 Μεθοδολογία γραμμής άνωσης και επίλυση του προβλήματος

Παρακάτω, φαίνεται η εξαγωγή των εξισώσεων του προβλήματος μας:

3.1 Τοπική εξίσωση άνωσης αεροτομής

Σύμφωνα με τη θεωρία λεπτών αεροτομών, η άνωση ανά μονάδα μήκους μιας διδιάστατης αεροτομής μπορεί να εκφραστεί μέσω της σχέσης Kutta–Joukowski:

$$L = \rho U \Gamma \tag{1}$$

Παράλληλα, η άνωση μπορεί να εκφραστεί σε όρους του συντελεστή άνωσης της αεροτομής:

$$L = \frac{1}{2} \rho U^2 c \, C_L \tag{2}$$

Για μικρές γωνίες προσβολής, ο συντελεστής άντωσης μεταβάλλεται γραμμικά με τη γωνία προσβολής και δίνεται από:

$$C_L = \frac{dC_L}{d\alpha} \left(\alpha_{\text{eff}} - \alpha_0 \right) \tag{3}$$

όπου α_{eff} είναι η αποτελεσματική γωνία προσβολής και α_0 η γωνία μηδενικής άνωσης της αεροτομής. Για λεπτές αεροτομές και μικρές γωνίες προσβολής, η κλίση της καμπύλης άνωσης λαμβάνεται ίση με $\frac{dC_L}{d\alpha} = 2\pi$, σύμφωνα με τη θεωρία λεπτών αεροτομών. Η τιμή αυτή προκύπτει από την υπόθεση ασυμπίεστης και ιδανικής ροής και εκφράζει τη γραμμική μεταβολή του συντελεστή άνωσης ως προς τη γωνία προσβολής.

3.2 Αποτελεσματική γωνία προσβολής

Στην περίπτωση πτέρυγας πεπερασμένου εκπετάσματος, η αποτελεσματική γωνία προσβολής διαφέρει από τη γεωμετρική λόγω της παρουσίας επαγόμενης ταχύτητας. Επιπλέον, στην περίπτωση ύπαρξης μεταπτερυγίου, η τοπική γωνία προσβολής επηρεάζεται και από τη γωνία κλίσης του flap.

Η αποτελεσματική γωνία προσβολής στο i-οστό σημείο ελέγχου γράφεται:

$$\alpha_{\text{eff},i} = \alpha + \delta_f + \frac{w_i}{U} \tag{4}$$

όπου w_i είναι η επαγόμενη κατακόρυφη ταχύτητα στο σημείο ελέγχου και δ_f η γωνία κλίσης του μεταπτερυγίου.

3.3 Σχέση κυκλοφορίας και επαγόμενης ταχύτητας

Η επαγόμενη ταχύτητα στο i-οστό σημείο ελέγχου προκύπτει από τη συνεισφορά όλων των δεσμευμένων και ουραίων στροβίλων της πτέρυγας και υπολογίζεται μέσω του νόμου Biot–Savart. Η σχέση γράφεται:

$$w_i = \sum_{j=1}^{N} \left(\mathbf{a}_j^i \cdot \mathbf{n}_i\right) \Gamma_j \tag{5}$$

όπου $\mathbf{a}_j^i$ είναι ο συντελεστής επιρροής του j-οστού στροβίλου στο i-οστό σημείο ελέγχου και $\mathbf{n}_i$ το μοναδιαίο διάνυσμα κάθετο στην τοπική επιφάνεια. Ο συντελεστής επιρροής $a_{i,j}$ υπολογίζεται από τον νόμο Biot–Savart ως:

$$a_{i,j} = \frac{1}{4\pi} \int \frac{d\boldsymbol{\ell} \times (\mathbf{r}_i - \mathbf{r})}{|\mathbf{r}_i - \mathbf{r}|^3}, \tag{6}$$

όπου το ολοκλήρωμα υπολογίζεται αναλυτικά για ευθύγραμμο vortex segment.

3.4 Τελική εξίσωση γραμμής άνωσης

Αντικαθιστώντας τις προηγούμενες σχέσεις και εξισώνοντας τις δύο εκφράσεις της άνωσης ανά μονάδα μήκους, προκύπτει για κάθε σημείο ελέγχου:

$$\rho U \Gamma_i = \frac{dC_l}{d\alpha} \left(\alpha + \delta_f - \alpha_0 + \frac{w_i}{U}\right) \frac{1}{2} \rho U^2 c \tag{7}$$

Αντικαθιστώντας την έκφραση της επαγόμενης ταχύτητας, η παραπάνω σχέση γράφεται:

$$\rho U \Gamma_i = \frac{dC_l}{d\alpha} \left(\alpha + \delta_f - \alpha_0 + \frac{1}{U} \sum_{j=1}^{N} \left(\mathbf{a}_j^i \cdot \mathbf{n}_i\right) \Gamma_j\right) \frac{1}{2} \rho U^2 c \tag{8}$$

Απλοποιώντας την παραπάνω εξίσωση για ασυμπίεστη ροή παίρνουμε τελικά:

$$\Gamma_i - \pi c_i \sum_{j=1}^{N} A_{ij} \Gamma_j = \pi U c_i \left(\alpha + \delta_f - \alpha_0\right) \tag{9}$$

Η παραπάνω εξίσωση εφαρμόζεται σε κάθε σημείο ελέγχου της πτέρυγας, οδηγώντας σε σύστημα γραμμικών εξισώσεων με αγνώστους τις κυκλοφορίες Γ_j, το οποίο επιλύεται αριθμητικά με τη μέθοδο Gauss-Elimination για εύρος γωνιών προσβολής και για διάφορες τιμές της γωνίας του flap.

4 Έλεγχος Ανεξαρτησίας Πλέγματος

Πριν από την εξαγωγή των αποτελεσμάτων, ελέγχθηκε η ανεξαρτησία του συντελεστή άνωσης , ως προς τον αριθμό των panels που βάζουμε ως input στον solver για την καλύτερη ακρίβεια των αποτελεσμάτων.

Για την ανεξαρτησία ελέγχθηκαν 2 περιπτώσεις σετ γωνίας προσβολής- γωνίας μεταπτερυγίου. Αυτές οι περιπτώσεις είναι:

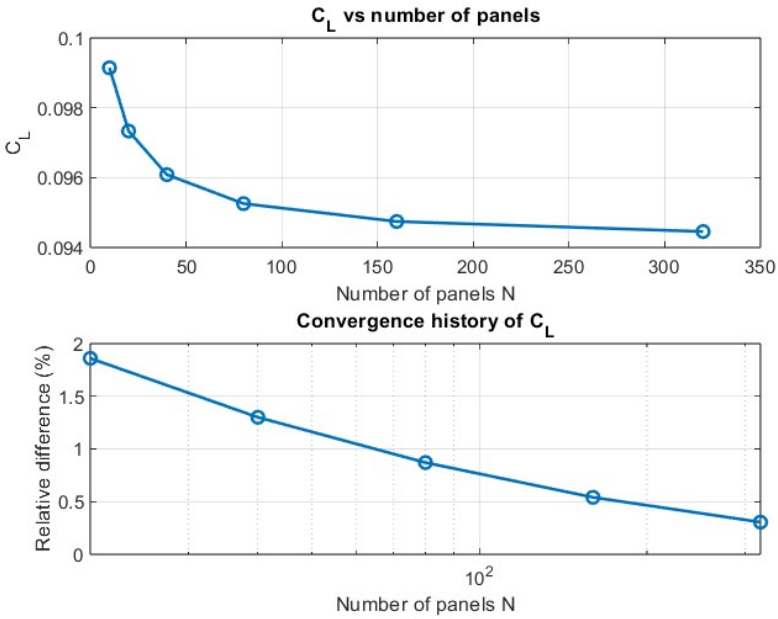
Περίπτωση (α): $\alpha = 0°,\ \delta_f = 5°$

Περίπτωση (β): $\alpha = 4°,\ \delta_f = 12°$

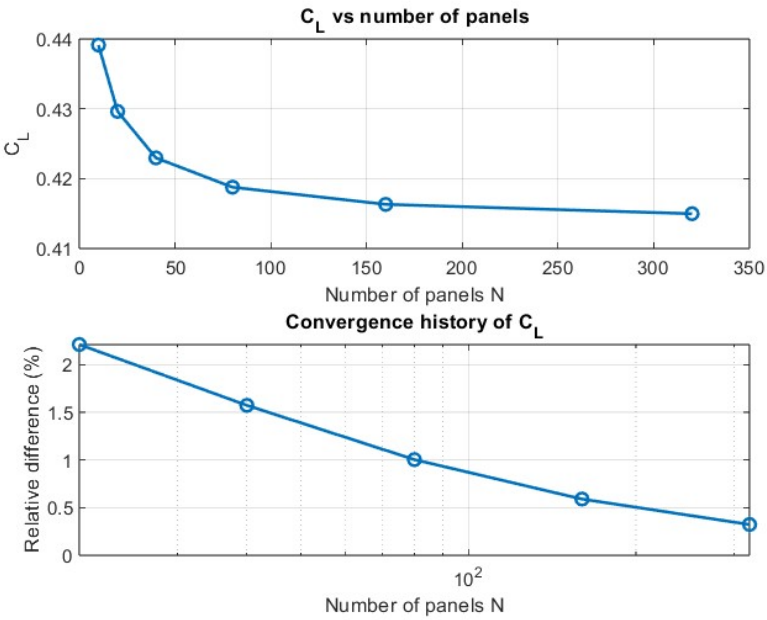
Ως κριτήριο σύγκλισης και για τις 2 αυτές περιπτώσεις τέθηκε το σχετικό σφάλμα του C_L να μην ξεπερνάει το 1%. Δηλαδή:

$$\varepsilon_{C_L} = \left| \frac{C_L^{(N)} - C_L^{(N_{\text{prev}})}}{C_L^{(N)}} \right| \times 100 < 1\%. \tag{10}$$

Παρακάτω φαίνονται τα αποτελέσματα της μελέτης των 2 αυτών περιπτώσεων:



Σχήμα 3: Σύγκλιση C_L για την περίπτωση (α)



Σχήμα 4: Σύγκλιση C_L για την περίπτωση (β)

Όπως φαίνεται, το κριτήριο σύγκλισης ικανοποιείται και για τις 2 περιπτώσεις για $N \geq 80$. Παρόλα αυτά, καθώς εξετάζουμε μόνο 2 από τις πολλές περιπτώσεις σετ γωνιών προσβολής-μεταπτερυγίου, για την εξαγωγή των αποτελεσμάτων θα τρέξουμε τον solver για $N = 160$ για να είμαστε σίγουροι για την ανεξαρτησία των αποτελεσμάτων σε κάθε περίπτωση.

5 Αποτελέσματα

Συνεχίζοντας, περνάμε στην επίλυση του συστήματος με $N = 160$ τον αριθμό panels ως input στον solver. Η επίλυση έγινε για το εύρος γωνιών και για τις τιμές γωνίας μεταπτερυγίου που φαίνονται παρακάτω:

- $\alpha \in [-4°,\ 20°]$, με βήμα $1°$
- $\delta_f = [\,-5°,\ 0°,\ 5°,\ 10°,\ 15°\,]$

Βρίσκοντας την κατανομή της κυκλοφορίας Γ από την επίλυση του γραμμικού συστήματος στη συνέχεια υπολογίζεται τελικά ο συντελεστής άνωσης C_L σύμφωνα με τον παρακάτω τύπο:

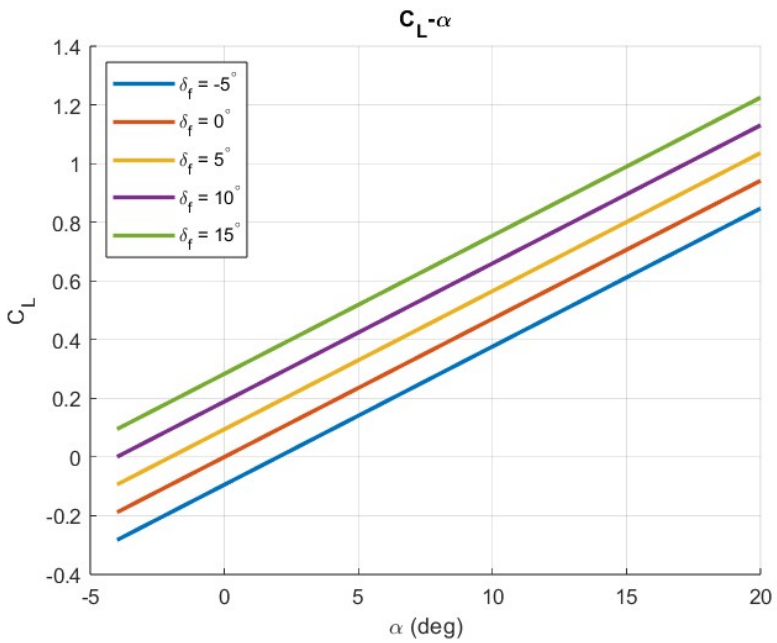
$$C_L = \frac{L}{\frac{1}{2}\rho U^2 S}. \tag{11}$$

όπου:

$$L = \rho U \int_{-b/2}^{b/2} \Gamma(y)\, dy, \tag{12}$$

το οποίο ολοκλήρωμα επιλύεται αριθμητικά με τη μέθοδο ορθογωνίου.

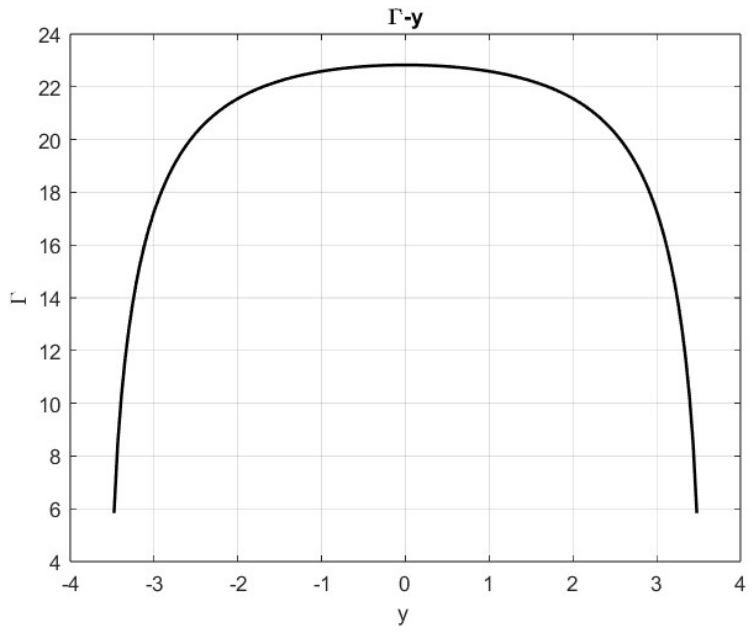
Τελικά παίρνουμε τα παρακάτω αποτελέσματα του C_L για κάθε γωνία μεταπτερυγίου:



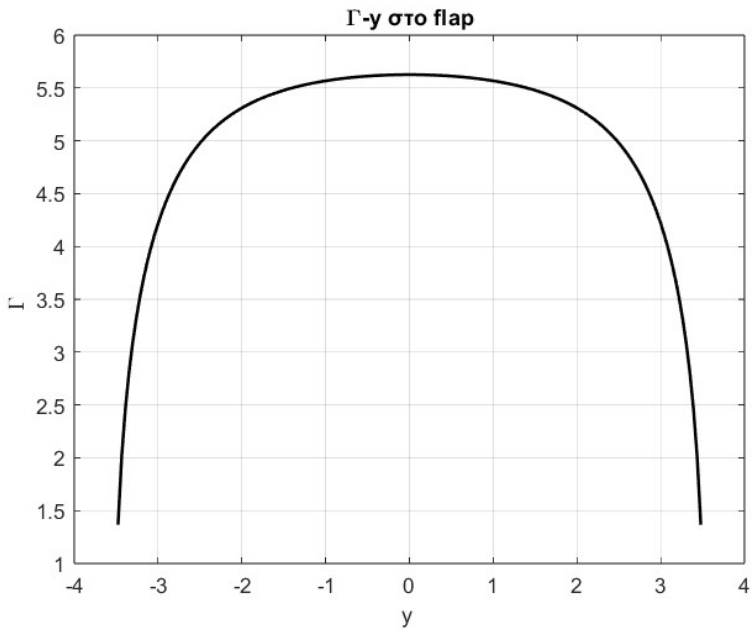
Σχήμα 5: Διάγραμμα C_L για τις διάφορες τιμές γωνίας μεταπτερυγίου

Όπως βλέπουμε, η επίδραση του flap είναι ότι μετακινεί παράλληλα την γραμμή του συντελεστή άνωσης, προς τα κάτω για $\delta_f < 0°$ ή προς τα πάνω για $\delta_f > 0°$. Συνεπώς, η κλίση της καμπύλης C_L δεν εξαρτάται από την γωνία του μεταπτερυγίου.

Ενδεικτικά, φαίνονται και τα διαγράμματα της κυκλοφορίας Γ για την περίπτωση όπου $\alpha = 10°$ και $\delta_f = 2°$ σε κάθε part της πτέρυγας:



Σχήμα 6: Κυκλοφορία στο main part της πτέρυγας



Σχήμα 7: Κυκλοφορία στο flap

6 Συμπεράσματα

Από τα αποτελέσματα προκύπτει ότι η επίδραση του μεταπτερυγίου εκδηλώνεται κυρίως ως παράλληλη μετατόπιση της καμπύλης C_L–α. Για θετικές τιμές δ_f ο συντελεστής άνωσης αυξάνεται για δεδομένη γωνία προσβολής, ενώ για αρνητικές τιμές μειώνεται.

Η κλίση της καμπύλης άντωσης C_{L_α} παραμένει πρακτικά αμετάβλητη για διαφορετικές γωνίες μεταπτερυγίου, γεγονός που υποδηλώνει ότι το flap δεν επηρεάζει την αεροδυναμική κλίση αλλά μεταβάλλει την αποτελεσματική γωνία προσβολής της πτέρυγας.

Όσον αφορά την κλίση αυτή, μπορούμε να ελέγξουμε την εγκυρότητα της από τον προσεγγιστικό τύπο που συνδέει την κλίση του C_L για μια 2D αεροτομή με την κλίση σε μια 3D πτέρυγα όπως στο πρόβλημα μας. Συγκεκριμένα:

$$C_{L,3D} = \frac{C_{L,2D}}{1 + \frac{2}{AR}}. \tag{13}$$

όπου $C_{L,2D} = 2\pi$ όπως είναι γνωστό από τη θεωρία.

Σύμφωνα με τον παραπάνω τύπο, η κλίση που θα αναμέναμε για το C_L υπολογίζεται ίση με $C_{L,3D} = 4.88 rad^{-1}$. Από το διάγραμμα που προέκυψε κατά την επίλυση, η κλίση υπολογίζεται ίση με $C_{L,3D} = 2.635 rad^{-1}$.

Παρατηρείται ότι η αριθμητικά υπολογισμένη κλίση είναι μικρότερη από την αντίστοιχη θεωρητική τιμή. Η απόκλιση αυτή αποδίδεται στις απλοποιήσεις της θεωρητικής σχέσης και στην πιο λεπτομερή περιγραφή της ροής που παρέχει ο αριθμητικός solver, ο οποίος λαμβάνει υπόψη τα επαγόμενα φαινόμενα και τη γεωμετρία του flap. Η θεωρητική τιμή χρησιμοποιείται αποκλειστικά ως σημείο αναφοράς και όχι ως ακριβής πρόβλεψη.

Α΄ Κώδικας Υπολογισμού (C++)

```

#include <cmath>
#include <iostream>
#include <fstream>
#include <algorithm>

using namespace std;

const double PI = 3.141592653589793;

int N = 160;
double c = 1.0;       // chord
double AR = 7.0;
double b = AR * c;    // span
double U = 100;
double Lwake = 10000.0 * c;


double induced_w_segment(
    double xp, double yp,
    double x1, double y1,
```

```
    double x2, double y2
) {
    const double EPS = 1e-12;

    double r1x = xp - x1;
    double r1y = yp - y1;
    double r2x = xp - x2;
    double r2y = yp - y2;

    double r1 = sqrt(r1x*r1x + r1y*r1y);
    double r2 = sqrt(r2x*r2x + r2y*r2y);

    if (r1 < EPS || r2 < EPS) return 0.0;

    double cross = r1x*r2y - r1y*r2x;
    if (fabs(cross) < EPS) return 0.0;

    double dot = (x2-x1)*(r1x/r1 - r2x/r2)
               + (y2-y1)*(r1y/r1 - r2y/r2);

    return (1.0 / (4.0 * PI)) * dot / cross;
}

int main() {


    double y1[700], y2[700], yc[700];
    for (int j = 0; j < N; j++) {
        y1[j] = -b/2 + j * b / N;
        y2[j] = -b/2 + (j+1) * b / N;
        yc[j] = 0.5 * (y1[j] + y2[j]);
    }



    double x_hinge =  0.75*c;
    double chord_main = x_hinge;
    double chord_flap = c- x_hinge;

    const int NCH = 2;
    const int M = N * NCH;


    double xbv[700], xcp[700], ycp[700];
    int surf[700];                      // 0 main, 1 flap
    int spanIdx[700];

    for (int j = 0; j < N; j++) {
        // main panel

```

```
        {
            int p = 0*N + j;
            surf[p] = 0;
            spanIdx[p] = j;
            double x_le = 0.0;
            xbv[p] = x_le + 0.25 * chord_main;
            xcp[p] = x_le + 0.75 * chord_main;
            ycp[p] = yc[j];
        }


        // flap panel
        {
            int p = 1*N + j;
            surf[p] = 1;
            spanIdx[p] = j;
            double x_le = x_hinge;
            xbv[p] = x_le + 0.25 * chord_flap;
            xcp[p] = x_le + 0.75 * chord_flap;
            ycp[p] = yc[j];
        }
    }




    // ---------------- influence matrix A
    static double A[700][700];
    static double A0[700][700];

    for (int i = 0; i < M; i++) {
        for (int j = 0; j < M; j++) {

            int js = spanIdx[j];

            double w = 0.0;

            // bound vortex of panel j
            w += induced_w_segment(
                xcp[i], ycp[i],
                xbv[j], y1[js],
                xbv[j], y2[js]
            );

            // wake leg 1
            w += induced_w_segment(
                xcp[i], ycp[i],
                xbv[j], y2[js],
                xbv[j] + Lwake, y2[js]
```

```
            );

            // wake leg 2
            w += induced_w_segment(
                xcp[i], ycp[i],
                xbv[j] + Lwake , y1[js],
                xbv[j]  , y1[js]
            );


            A0[i][j] = w;
        }
    }

    // ---------------- flap parameters ----------------FLAP


    double alpha0 = 0.0;

    double delta_f_deg = 15.0;                ////////// ΓΩΝΙΑ ΦΛΑΠ
    double delta_f = delta_f_deg * PI/180.0;

    // flap span region   (      )
    double y_flap_start = 0.0 * (b/2.0);
    double y_flap_end   = 1.0 * (b/2.0);




    // ---------------- outputs ----------------
    ofstream fout("CL_vs_alpha_5.txt");
    fout << "% alpha_deg   CL\n";

    ofstream fg("Gamma_distribution.txt");
    fg << "% alpha_deg  p  surf  y  Gamma\n";



    //LOOP GWNIWN



    static double bvec[700];
    static double Gamma[700];

    for (double alpha_deg = -4.0; alpha_deg <= 20.0; alpha_deg += 1.0) {

        double alpha = alpha_deg * PI / 180.0;

```

```
    for (int i = 0; i < M; i++) {

        int js = spanIdx[i];
        double yabs = fabs(yc[js]);


        double ci = (surf[i] == 0 ? chord_main : chord_flap);


        double Ki = PI * ci;


        double alpha_i = (alpha - alpha0);

        if (surf[i] == 1) {
            if (yabs >= y_flap_start && yabs <= y_flap_end) {
                alpha_i += delta_f;
            }
        }

        // RHS
        bvec[i] = Ki * U * alpha_i;




        for (int j = 0; j < M; j++) {
            A[i][j] = -Ki * A0[i][j];
        }


        A[i][i] += 1.0;
    }









        // EPILYSH SYSTHMATOS KAI META CL


        // -------- Gaussian elimination --------
```

```
        for (int k = 0; k < M; k++) {

            int piv = k;
            double amax = fabs(A[k][k]);
            for (int i = k+1; i < M; i++) {
                if (fabs(A[i][k]) > amax) {
                    amax = fabs(A[i][k]);
                    piv = i;
                }
            }
            if (amax < 1e-14) {
                cerr << "Singular/ill-conditioned matrix at alpha=" << alpha_deg << " deg\n";
                return 1;
            }
            if (piv != k) {
                for (int j = k; j < M; j++) std::swap(A[k][j], A[piv][j]);
                std::swap(bvec[k], bvec[piv]);
            }

            for (int i = k+1; i < M; i++) {
                double m = A[i][k] / A[k][k];
                for (int j = k; j < M; j++)
                    A[i][j] -= m * A[k][j];
                bvec[i] -= m * bvec[k];
            }
        }

        for (int i = M-1; i >= 0; i--) {
            double s = bvec[i];
            for (int j = i+1; j < M; j++)
                s -= A[i][j] * Gamma[j];
            Gamma[i] = s / A[i][i];
        }




        // ------------------------------------------------------------

        // save Gamma
        for (int p = 0; p < M; p++) {
            int js = spanIdx[p];
            fg << alpha_deg << "  " << p << "  " << surf[p] << "  " << yc[js] << "  " << Gamma[p] << "\n";
        }
        fg << "\n";


```

```

        // CL


        double sumG = 0.0;
        for (int j = 0; j < N; j++) {
            int p_main = 0*N + j;
            int p_flap = 1*N + j;
            double dyj = y2[j] - y1[j];
            sumG += (Gamma[p_main] + Gamma[p_flap]) * dyj;
        }
        double S = b * c;
        double CL = 2.0 * sumG / (U * S);

        cout << alpha_deg << "  " << CL << endl;
        fout << alpha_deg << "  " << CL << endl;
    }




    fout.close();
    fg.close();
    return 0;
}
```